UNIX for Devs h_da - Exit Vim CMD guide

how to exit vim on cmd/

To **exit Vim** from the command line (CMD), here are the main options depending on what you want to do:

🟢 If you haven't made changes

Type:

```
less

:q
```

Then press **Enter**.
👉 This means "quit".

🟡 If you have made changes but don't want to save them

Type:

```
less

:q!
```

Then press **Enter**.
👉 The `!` forces Vim to quit **without saving**.

🔵 If you want to save and exit

Type:

```
ruby

:wq
```

Then press **Enter**.
👉 This means "write and quit".

Alternatively, you can use:

```ruby
:x
```

Same effect — saves and exits.

⚙️ Quick combo

If you're stuck in insert mode (typing text and nothing works), first press:

```nginx
Esc
```

to return to command mode, **then** type one of the above commands (like `:q!`, `:wq`, etc.).

Would you like me to also show how to exit **Vim from insert mode or visual mode** step by step (in case you get stuck)?

how to activate line number

To **show line numbers in Vim**, follow these simple steps 👇

▯ Temporarily (for current session only):

1. Press **`Esc`** (to make sure you're in *normal mode*).
2. Type:

```arduino
:set number
```

3. Press **Enter**.

✅ Line numbers will now appear on the left side of your file.

⚙️ To make it permanent (always show line numbers):

1. Open (or create) your Vim configuration file:

```bash
vim ~/.vimrc
```

2. Add this line:

```
arduino

set number
```

3. Save and exit (`:wq`).

Now Vim will always start with line numbers enabled.

🧩 Extra tip:

If you prefer *relative* line numbers (useful for navigation):

```
arduino

:set relativenumber
```

Or both together:

```
arduino

:set number relativenumber
```

Would you like me to explain what *relative line numbers* do and why many developers use them?



strgt

Could you clarify what you mean by **“strgt”**?
It might be a typo — did you mean something like:

- `:wq` (save and quit in Vim)?
- Or maybe **“straight”**, **“string”**, or **“shortcut”**?

If you’re referring to a **keyboard shortcut** (like Ctrl or something in Vim or CMD), please specify — I’ll explain it clearly.



strgt + R whats strgt

Ah — you probably mean **`Strg + R`**!

✅ On **German (and some European) keyboards**, `Strg` stands for **"Steuerung"**, which is the **Control key** — the same as `Ctrl` on English keyboards.

So:

```
css

Strg + R  →  Ctrl + R
```

💡 In context:

- In **Windows CMD**, `Ctrl + R` doesn't do much by default (it may recall previous commands in some shells).
- In **Vim**, `Ctrl + R` is used to **redo** (undo the last undo).
- In **bash / zsh**, `Ctrl + R` opens **reverse search** in your command history — super useful for finding old commands.

Would you like me to explain what `Ctrl + R` does specifically in **Vim** or in the **Windows Command Prompt**?



how to save

Here's how to **save your file in Vim** 👇

💾 Basic save command

1. Press **`Esc`** (to leave insert mode).
2. Type:

   ```
   ruby

   :w
   ```

3. Press **Enter**.

✅ This writes (saves) the file to disk.

🧠 Save and quit at once

```
ruby

:wq
```

or simply

```ruby
:x
```

Both save your changes and exit Vim.

⚠️ If the file is read-only (permission denied)

You can **force save with sudo privileges**:

```bash
:w !sudo tee %
```

Then press **Enter**, type your password, and it'll save.

Quick summary

Action	Command	Description
Save file	`:w`	Writes changes
Save and quit	`:wq` or `:x`	Writes and exits
Quit without saving	`:q!`	Discards changes

Would you like me to show how to **save under a different filename** (like "Save As")?



macros

Great question — **macros** in Vim are one of its most powerful features ⚙️

They let you **record a sequence of commands** and **replay them** — super useful for repeating edits efficiently.

🎬 Recording a macro

1. Press **Esc** (to be sure you're in *normal mode*).
2. Type:

```
perl

q<letter>
```

Example:

```
nginx

qa
```

➜ This starts recording a macro into register **a**.

3. Perform any series of Vim commands you want (move, delete, insert, etc.).
4. When done, press:

```
css

q
```

➜ This stops recording.

▶️ Playing (replaying) a macro

To run the macro you recorded in register **a**:

```
css

@a
```

You can also repeat it multiple times, for example:

```
css

10@a
```

➜ Runs it 10 times.

💡 Pro tips

- Use `@@` to **replay the last macro** again.
- You can view or edit the contents of a macro by typing:

  ```
  ruby

  :registers
  ```

- You can also record into **any register** (a–z).

🧠 Example

Let's say you want to **add a semicolon at the end of every line**:

1. Move to the first line.
2. Type:

```
php-template

qaA;<Esc>j
```

   (This means: record to **a**, go to end of line, insert **;**, exit insert mode, move down.)

3. Stop recording:

```
css

q
```

4. Then run:

```
css

@a
```

   repeatedly — or use

```
css

100@a
```

   to do it for 100 lines.

Would you like me to show you how to **save and reuse macros across sessions** (so they don't disappear when you close Vim)?